KPR Institute of Engineering and Technology
(Autonomous)
Avinashi Road, Arasur, Coimbatore - 641 407

U21ADP05 – EXPLORATORY DATA ANALYSIS AND VISUALIZATION

III year B.Tech. Artificial Intelligence and Data Science

Assignment II – Case Study Report

Teaching a Machine to Read Handwriting: Exploratory Analysis and a Convolutional Neural Network for Optical Digit Recognition

Vinith A (25ADL10)

Academic Year: 2026 - 27

Abstract

Banks, post offices and examination cells still process large volumes of handwritten forms, so reliable automatic digit recognition saves time and reduces errors. This case study examines the UCI Optical Recognition of Handwritten Digits dataset: 5,620 digits from 43 writers, each reduced to 64 pixel features. The data were audited, the original 32 x 32 bitmaps were decoded to verify the feature extraction, and the images were explored through class balance, pixel-value and ink distributions, pixel correlation, PCA, t-SNE and anomaly detection. A convolutional neural network was tuned over eight configurations and tested on 1,797 digits from 13 writers it had never seen. It reached 99.05% accuracy with only 17 errors, significantly better than k-nearest neighbours (97.55%), with well-calibrated confidence.

Keywords: handwritten digit recognition; image data; exploratory data analysis; binomial mixture; t-SNE; convolutional neural network; occlusion sensitivity; calibration.

Table of Contents

1 Introduction and Objective 1
1.1 Motivation 1
1.2 Problem Statement 1
1.3 Objectives 1
1.4 Related Work 2
1.5 Report Structure 2
2 Dataset Description 3
2.1 Source and Licence 3
2.2 Size and Fields 3
2.3 Basic Statistics 3
3 EDA and Preprocessing 4
3.1 Acquire and Parse: Verifying the Features 4
3.2 Missing, Invalid and Duplicate Records 4
3.3 Outliers 5
3.4 Transformation and Encoding 5
3.5 Train, Validation and Test Sets 5
3.6 Insights from Preprocessing 5
4 Data Visualization 6
4.1 The Seven Stages of Visualization 6
4.2 Class Balance 6
4.3 What an Average Digit Looks Like 7
4.4 Discrete Distribution of Pixel Values 7
4.5 Continuous Distribution of Total Ink 8
4.6 Pixel Variability and Correlation 9
4.7 Shape Features for Confusable Digits 10
4.8 Multivariate Structure: PCA and t-SNE 11
4.9 Unusual Handwriting 11
4.10 Do the Two Groups of Writers Differ? 12
4.11 Interact: the Digit Lab 13
4.12 Summary of the Exploratory Findings 14
5 Deep Learning Model 15
5.1 Why a Convolutional Network 15
5.2 Architecture 15
5.3 Training Parameters 15
5.4 Hyperparameter Tuning 16
5.5 Feature Analysis: What the Network Looks At 17
5.6 Baselines 17
6 Result Visualization and Interpretation 18
6.1 Loss and Accuracy versus Epoch 18
6.2 Confusion Matrix 19
6.3 Precision, Recall and F1 by Digit 20
6.4 ROC Curves and AUC 20
6.5 The Mistakes 21
6.6 Confidence, Calibration and the Error Distribution 21
6.7 What the Network Learned 22
6.8 Comparison with the Baselines 23

7 Conclusion and Future Scope 24
7.1 What Was Learned from the Data 24
7.2 Model Performance 24
7.3 Challenges Faced 24
7.4 Future Scope 24
References 26
Appendix A: Repository and Reproduction 27
Appendix B: Source Code (Key Sections) 28

List of Figures

Fig. 3.1 Original 32 x 32 bitmaps (top, pink grid = 4 x 4 blocks) and the derived 8 x 8 counts (bottom). 4
Fig. 4.1 Share of each digit among training and test images 6
Fig. 4.2 Mean image (top) and pixel standard deviation (bottom) for each digit, training set 7
Fig. 4.3 Distribution of the 0-16 pixel counts with binomial and binomial-mixture fits, and the probability of ink at each position.....7
Fig. 4.4 Total ink per image: histogram with normal and skew-normal fits, normal Q-Q plot and ink by digit 8
Fig. 4.5 Standard deviation of each pixel and the Spearman correlation matrix of the 62 non-constant pixels 9
Fig. 4.6 Ink position for digits 1, 7 and 4, and the left-right ink balance of 3, 5 and 8.....10
Fig. 4.7 Cumulative variance of the principal components, the first two components and a t-SNE map; each point is drawn as its digit 11
Fig. 4.8 Isolation-forest anomaly scores and the twelve most atypical training images..... 11
Fig. 4.9 Mean training and test images and their difference (stars: significant after FDR correction) ...12
Fig. 4.10 The Digit Lab page (dashboard/index.html): draw-a-digit panel and test-set explorer.....13
Fig. 5.1 Validation accuracy of the eight configurations.....16
Fig. 5.2 Occlusion sensitivity: drop in the probability of the true digit when one pixel is erased.....17
Fig. 6.1 Training and validation loss (log scale) and accuracy for each epoch 18
Fig. 6.2 Confusion matrix of the CNN on the writer-independent test set.....19
Fig. 6.3 Precision, recall and F1 score for each digit..... 20
Fig. 6.4 One-versus-rest ROC curves of the ten digits with micro-averages for the CNN and k-NN..... 20
Fig. 6.5 All misclassified test images, sorted by the network's confidence.....21
Fig. 6.6 Reliability diagram and confidence of correct and incorrect predictions 21
Fig. 6.7 t-SNE map of the network's 64-dimensional embedding for the test images (pink: errors)..... 22
Fig. 6.8 Test accuracy of the CNN and the two baselines..... 23

List of Tables

Table 2.1 – Composition of the dataset 3
Table 2.2 – Summary statistics for ten representative pixels (training set; full table in results/pixel_summary.csv) 3
Table 3.1 – Data-quality audit 4
Table 3.2 – Transformations 5
Table 4.1 – Fry's seven stages in this project 6
Table 4.2 – Discrete models for a block count (16 trials per block) 7
Table 4.3 – Summary of the statistical analysis 14
Table 5.1 – Architecture of the selected CNN 15
Table 5.2 – Training configuration 15
Table 5.3 – Grid search over filters, dropout and augmentation (selected: #6) 16
Table 6.1 – Test-set performance 23

1 Introduction and Objective

1.1 Motivation

Handwritten numbers are everywhere in everyday administration: PIN codes on envelopes, amounts on cheques, register numbers on answer sheets and readings on paper forms. Reading them by hand is slow and error-prone, which is why optical character recognition was one of the earliest practical successes of machine learning. Digit recognition remains a useful case study because it is an image problem with a clear ground truth, so every exploratory finding can be checked against what the eye sees.

This study uses the optical digit data collected at Bogazici University (Alpaydin and Kaynak [1], [2]). Writers filled in preprinted forms that were scanned and normalised with the NIST form-processing software [4]. Each digit became a 32 x 32 black-and-white bitmap, which was then summarised as an 8 x 8 grid of counts. The dataset is unusual in one helpful way: the people who wrote the training digits are different from the people who wrote the test digits, so the test measures how well a model reads a stranger's handwriting.

1.2 Problem Statement

Given the 64 block counts of one handwritten digit, identify which of the ten digits it is. Before modelling, the study asks what the images look like statistically: how balanced the classes are, how pixel values and total ink are distributed, which regions of the grid carry information, how strongly neighbouring pixels depend on each other, whether the ten digits form natural groups, which samples are unusual, and whether the two groups of writers differ.

1.3 Objectives

1. Acquire and parse the original UCI files, including the compressed 32 x 32 bitmaps, and verify how the 64 features were produced.
2. Audit data quality: missing and out-of-range values, duplicates, leakage between training and test writers, constant pixels and outliers.
3. Describe single variables and fit discrete and continuous probability models to pixel values and total ink.
4. Study relationships between pixels and between shape features, and explore the multivariate structure with PCA and t-SNE.
5. Present at least five purposeful visualizations covering all seven stages of Fry's process, including an interactive page.
6. Build and tune a convolutional neural network, explain what it attends to, and evaluate it on unseen writers against simpler baselines.

1.4 Related Work

LeCun et al. [5] showed that convolutional networks, which share small learned filters across an image, read handwritten digits far better than hand-engineered features, and their LeNet design is the ancestor of the network used here. Two later training practices are adopted directly. Batch normalisation [6] stabilises training, and small random shifts of the training images [7] teach the network that a digit does not change when it moves slightly. For explanation, Zeiler and Fergus [8] introduced occlusion analysis: hide part of the input and observe how the prediction changes. Section 5.5 applies that idea pixel by pixel.

On the exploratory side, t-SNE [9] has become the standard way to draw high-dimensional image data in two dimensions, and the isolation forest [10] isolates unusual samples without assuming any distribution. Modern networks are often over-confident, so their calibration should be checked [11]. Comparisons between two classifiers on the same test images call for a paired test such as McNemar's [12]. Alpaydin and Kaynak's own work on this dataset [3] combined simple and complex classifiers. The k-nearest-neighbour rule [13] serves as the classical baseline in this report.

1.5 Report Structure

Section 2 describes the dataset and Section 3 the audit and preprocessing. Section 4 presents the visual and statistical exploration organised by Fry's seven stages [14]. Section 5 builds and tunes the CNN, and Section 6 evaluates it on the unseen writers. Section 7 concludes. The appendices give the repository layout and the main source code.

2 Dataset Description

2.1 Source and Licence

Optical Recognition of Handwritten Digits, UCI Machine Learning Repository, dataset 80, donated in 1998 by E. Alpaydin and C. Kaynak [1] (DOI 10.24432/C50P49, licence CC BY 4.0). The archive (0.6 MB) contains the preprocessed tables `optdigits.tra` and `optdigits.tes` and four files of original 32 x 32 bitmaps compressed with the Unix `compress` program (.Z). The project reads every file directly from the unmodified zip archive.

2.2 Size and Fields

Table 2.1 – Composition of the dataset

Item	Training file	Test file
Images	3,823	1,797
Writers	30	13 (different people)
Fields	64 pixel counts + 1 class label	64 pixel counts + 1 class label
Pixel values	integers 0-16	integers 0-16
Classes	digits 0-9	digits 0-9

Each of the 64 fields (named `p<row><column>` in this project, from `p00` at the top left to `p77` at the bottom right) counts how many of the 16 pixels in one 4 x 4 block of the original bitmap are black. A value of 0 means empty paper, 16 means a solid stroke, and values in between mark the edge of a stroke. Together with the label, there are 65 fields per record, well above the 15 required.

2.3 Basic Statistics

Table 2.2 – Summary statistics for ten representative pixels (training set; full table in results/pixel_summary.csv)

Pixel	Mean	SD	Median	Q3	Max	Skewness	Zeros (%)
p02	5.48	4.63	5	9	16	0.48	21.9
p13	11.71	4.01	13	15	16	-0.79	1.1
p20	0.01	0.12	0	0	5	30.13	99.8
p33	9.13	5.90	10	15	16	-0.30	14.6
p34	9.67	6.28	12	16	16	-0.49	19.1
p36	2.34	3.62	0	4	16	1.50	59.4
p43	9.24	6.19	11	16	16	-0.34	18.4
p52	6.46	6.50	4	13	16	0.33	37.8
p63	9.89	5.14	10	15	16	-0.38	5.5
p66	3.74	4.90	1	7	16	1.13	48.0

Across all 359,680 pixel values, 48.8% are 0 and 9.2% are 16. Edge pixels such as `p20` and `p63` are almost always empty and strongly right-skewed. Central pixels such as `p33` and `p34` have medians near the top of the scale and are left-skewed. The summary statistics of a single pixel therefore depend mainly on where that pixel sits in the frame.

3 EDA and Preprocessing

3.1 Acquire and Parse: Verifying the Features

Python cannot open `.Z` files with its standard library, so a small LZW decoder was written (`common.unlzw`). Its output was checked against the reference `gzip` tool and matched byte for byte on all four files. Rebuilding the 8 x 8 counts from the decoded bitmaps of the test writers reproduces `optdigits.tes` exactly for 100% of the 1,797 images, labels included. The features are therefore exactly what the documentation says they are.

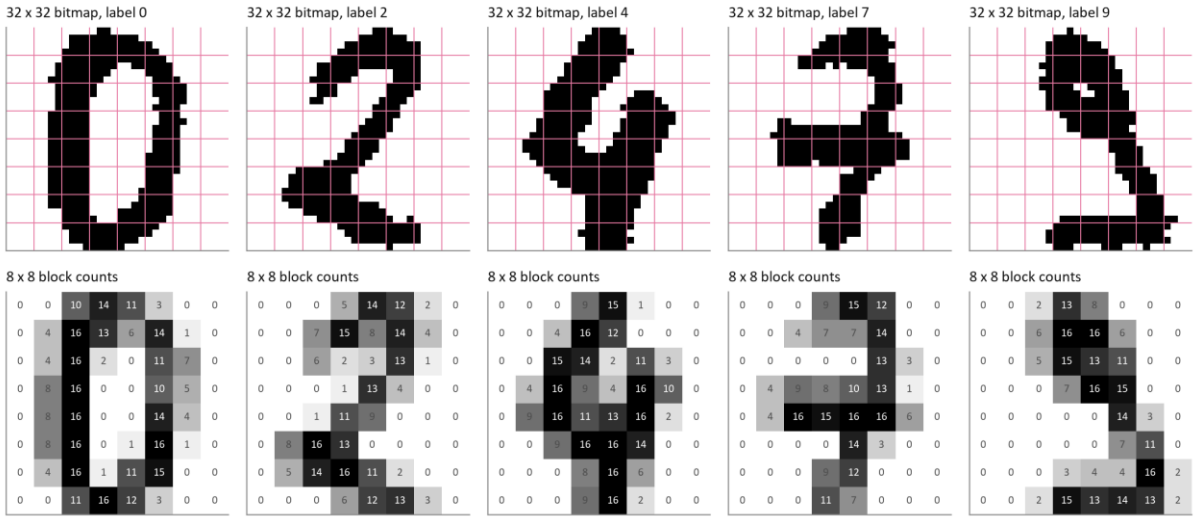


Fig. 3.1 Original 32 x 32 bitmaps (top, pink grid = 4 x 4 blocks) and the derived 8 x 8 counts (bottom)

What it shows	Five test digits as scanned bitmaps with the 4 x 4 block grid overlaid, and the 8 x 8 count matrix produced from each one, with every count printed in its cell.
Why it was created	Parsing is the second of Fry's stages. Seeing the raw and the processed form side by side makes the meaning of every feature concrete and confirms the preprocessing.
Insight obtained	Reducing 1,024 bits to 64 counts keeps the shape of every digit while discarding detail such as stroke texture. Grey cells (counts between 1 and 15) mark where a stroke crosses a block boundary, so the counts also encode stroke thickness and position within each block.

3.2 Missing, Invalid and Duplicate Records

Table 3.1 – Data-quality audit

Check	Result
Missing cells (training and test)	0
Values outside 0-16	0
Duplicate rows within training / test	0 / 0
Identical images shared by training and test	0
Identical images with conflicting labels	0
Zero-variance pixels (training)	p00, p47
Pixels empty in more than 99% of images	p00, p10, p20, p30, p37, p40, p47, p60, p70

The data are clean in the usual sense: no gaps, no impossible values and no duplicates. The audit matters for two less obvious reasons. First, no image appears in both training and test files, so test

results cannot be inflated by leakage. Second, two pixels (`p00` and `p47`) never change in the training set and seven more are almost always empty. For tabular models such pixels are useless and can be removed. For the CNN they were kept, because the network needs the complete 8 x 8 grid to keep its spatial structure.

3.3 Outliers

For image data an outlier is an unusual way of writing a digit, not a single extreme number. The z-score and interquartile rules are poorly suited here because every pixel is bounded between 0 and 16. Atypical images were therefore identified with an isolation forest [10] on all 64 features (Section 4.9). It flagged 94 training images (2.46%) beyond Tukey's fence on the anomaly score. Inspection showed that they are genuine but unusual handwriting, mostly open-topped 4s and 7s written with a crossbar. They were kept, because removing hard examples would make the model weaker on exactly the writers it finds difficult.

3.4 Transformation and Encoding

Table 3.2 – Transformations

Step	Applied to	Details
Scaling	all 64 pixels	x / 16, mapping counts to the interval [0, 1] (min-max scaling with known limits)
Reshaping	all images	64-vector to an 8 x 8 x 1 tensor (row-major order), the input shape of the CNN
Label encoding	class label	integer 0-9 used directly with sparse categorical cross-entropy (equivalent to one-hot)
Augmentation	training batches only	random shifts of up to one pixel in each direction (tuned on/off)
Baseline inputs	k-NN, logistic regression	the same scaled 64-vector

3.5 Train, Validation and Test Sets

The split follows the original authors. The 3,823 images of the 30 training writers were divided by stratified sampling into 3,249 training and 574 validation images (85 / 15), so every digit keeps its share. The 1,797 images of the 13 test writers form the test set. It was used once, after all modelling decisions were final. Because no test writer appears in training, the test accuracy estimates performance on new people rather than on new samples from familiar ones.

3.6 Insights from Preprocessing

- The dataset is complete and leak-free, and the 64 features were reproduced exactly from the raw bitmaps.
- About half of all pixel values are zero; the informative pixels lie in columns 1 to 6 and rows 0 to 7.
- Outliers are unusual handwriting styles, not errors, so they were kept and studied rather than deleted.

4 Data Visualization

Every visual below answers one question about the data. Image data call for image-style displays (sample grids, mean images, heat maps of pixel statistics) alongside classical charts. In all charts, green marks the main series and pink the comparison series.

4.1 The Seven Stages of Visualization

Table 4.1 – Fry's seven stages in this project

Stage [14]	In this project
1 Acquire	Original UCI zip archive read in place; no manual editing
2 Parse	LZW decoder for the .Z bitmaps; 64-vectors reshaped to 8 x 8 images; features verified (Fig. 3.1)
3 Filter	Audit table, constant-pixel check, train-test leakage check, isolation-forest outliers
4 Mine	Binomial mixture by EM, normal and skew-normal fits, Kruskal-Wallis, Spearman, PCA, t-SNE, CNN
5 Represent	Image grids, mean and spread images, heat maps, distribution plots, scatter and embedding maps
6 Refine	Log scales, values printed in cells, glyph markers instead of ten colours, consistent colour roles
7 Interact	Digit Lab web page: draw a digit, browse and filter the test predictions (Section 4.11)

4.2 Class Balance



Fig. 4.1 Share of each digit among training and test images

What it shows	The percentage of images belonging to each digit, separately for the training and the test writers, with the 10% line for a perfectly balanced set.
Why it was created	Class balance decides whether accuracy is a fair headline metric and whether class weighting is needed.
Insight obtained	The classes are almost perfectly balanced: training counts range from 376 to 389 and test counts from 174 to 183. Chi-square tests find no departure from uniformity (training chi-square = 0.66, p = 1.000) and no difference between the two groups of writers (chi-square = 0.3, p = 1.000). Plain accuracy is therefore a fair metric and no re-weighting is needed. Note the zoomed vertical axis, which starts at 8.5%.

4.3 What an Average Digit Looks Like

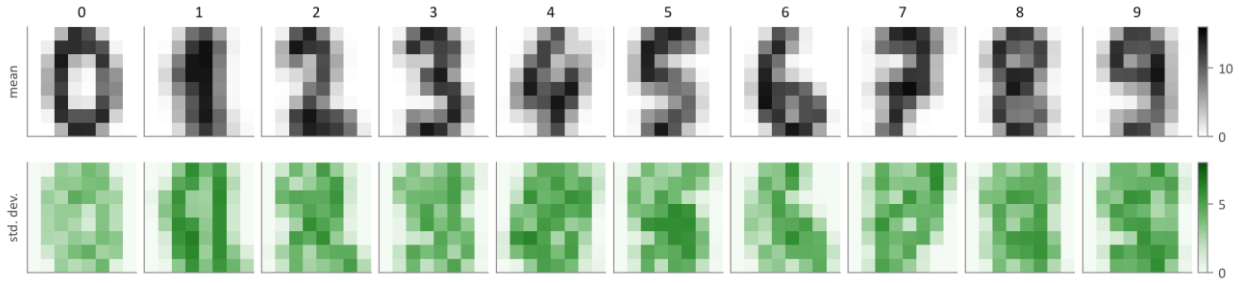


Fig. 4.2 Mean image (top) and pixel standard deviation (bottom) for each digit, training set

What it shows	For every digit, the average of all its training images and a map of how much each pixel varies between writers.
Why it was created	Mean images show the prototype the data define for each class. The spread maps show where writers disagree, which is exactly where a classifier will find the task hard.
Insight obtained	Every prototype is clearly recognisable, which promises high accuracy. The spread is lowest where strokes are nearly universal, such as the vertical bar of a 1 and the loop of a 0, and highest along stroke edges and on optional parts: the foot of a 1, the crossbar of a 7 and the open or closed top of a 4. The 3, 5, 8 and 9 prototypes overlap heavily in the right half, which anticipates the confusions in Section 6.

4.4 Discrete Distribution of Pixel Values

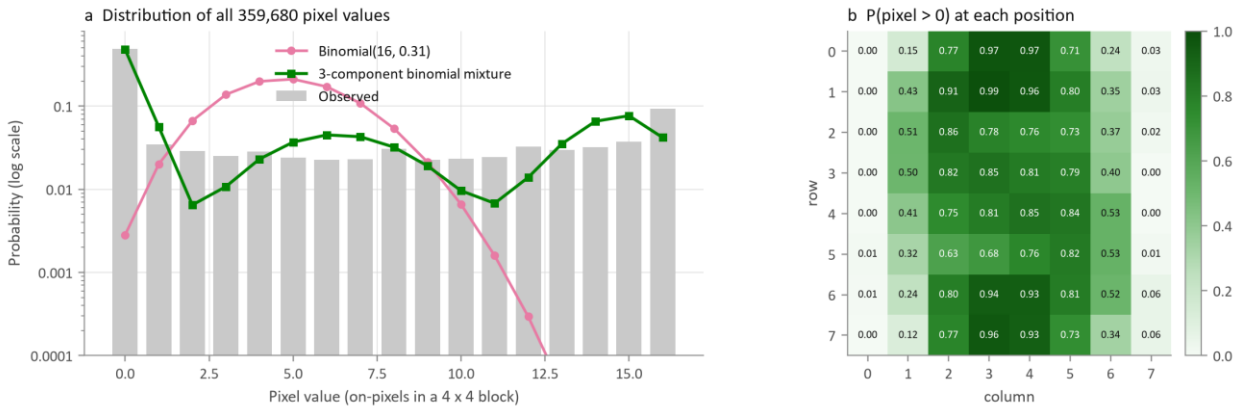


Fig. 4.3 Distribution of the 0-16 pixel counts with binomial and binomial-mixture fits, and the probability of ink at each position

Table 4.2 – Discrete models for a block count (16 trials per block)

Model	Parameters	AIC
Binomial(16, p)	p = 0.307	5,061,905
Mixture of three binomials (EM)	weights 0.54 / 0.23 / 0.24; p = 0.007 / 0.400 / 0.897	1,574,236

What it shows	Panel (a) is the probability of each block count from 0 to 16 on a log scale, with a single binomial model and a three-component binomial mixture fitted by the EM algorithm. Panel (b) shows, for each position, the share of images in which that block contains any ink.
Why it was created	Each block count is literally the number of black pixels among 16, so the binomial is the natural first model. Testing it reveals whether one 'inking probability' can describe every block.
Insight obtained	One binomial fails badly: the observed distribution is U-shaped, with a huge spike at 0, a second peak at 16 and a flat middle, while the binomial predicts a hump around 5. The three-component mixture cuts the AIC by 3.5 million and has a clear physical reading: 54% of blocks are background (p = 0.007), 23% lie on stroke edges (p = 0.40) and 24% lie inside thick strokes (p = 0.90). It still underestimates the middle values, because real edge coverage is spread almost evenly rather than clustered. Panel (b) confirms that columns 0 and 7 are nearly always empty while the centre of the grid is inked in 75-99% of images.

4.5 Continuous Distribution of Total Ink

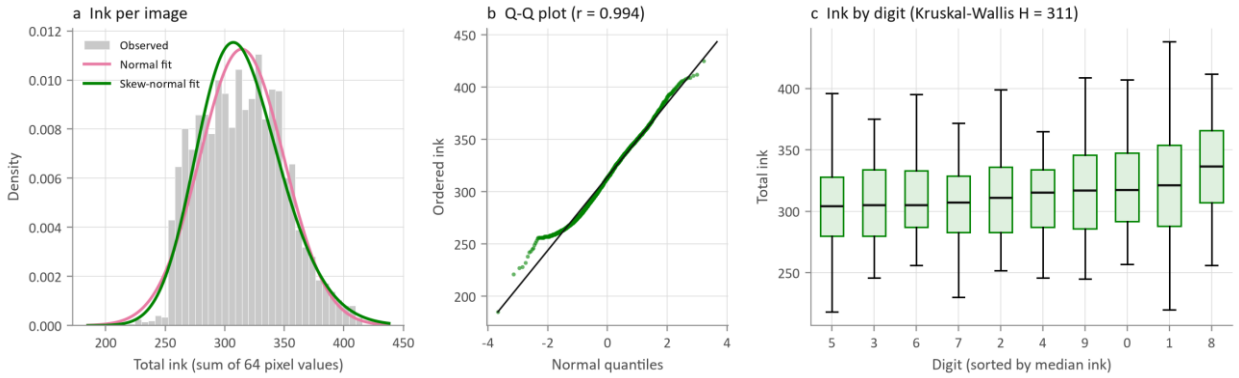


Fig. 4.4 Total ink per image: histogram with normal and skew-normal fits, normal Q-Q plot and ink by digit

What it shows	The sum of all 64 counts, a measure of how much ink a digit uses, for all 5,620 images: its histogram with two fitted densities, a normal quantile plot, and box plots by digit sorted by median.
Why it was created	Total ink is the most natural continuous summary of an image. Its distribution tells whether normal-theory methods apply, and its differences between digits show whether a simple feature already separates classes.
Insight obtained	Total ink averages 314 (SD 35) and is close to normal (Q-Q correlation 0.994), but slightly right-skewed (skewness 0.24): a Shapiro-Wilk test on 1,000 images rejects normality (W = 0.989) and the skew-normal lowers the AIC from 56,054 to 55,992. The Q-Q plot also shows a bend at the lower end, where thin, faint digits pile up. Digits differ in ink (Kruskal-Wallis H = 311, $p < 10^{-60}$) but only slightly (epsilon-squared = 0.055): 8 uses the most (median 336) and 5 the least (304). The overlap between the boxes shows that ink alone cannot classify digits, because size normalisation in preprocessing makes every digit fill the frame.

4.6 Pixel Variability and Correlation

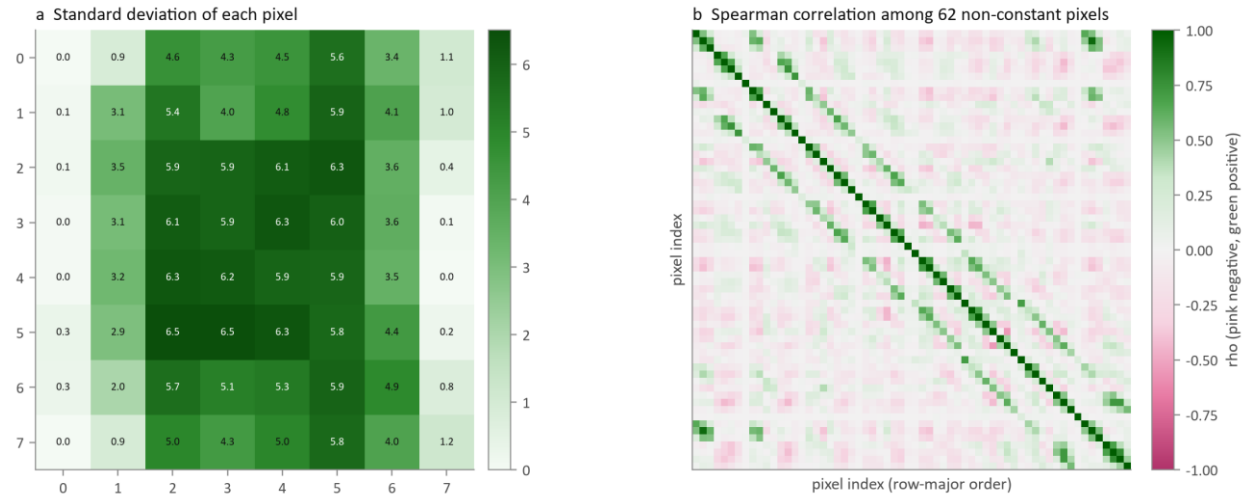


Fig. 4.5 Standard deviation of each pixel and the Spearman correlation matrix of the 62 non-constant pixels

What it shows	Panel (a) maps the standard deviation of every pixel. Panel (b) is the 62 x 62 Spearman correlation matrix in row-major pixel order on a diverging pink-green scale.
Why it was created	Variability shows where information lives; correlation shows how much of it is redundant. Both matter for choosing a model: strong local correlation is precisely the structure convolution exploits.
Insight obtained	Variability is concentrated in the six inner columns, peaking near the centre (SD up to 6.5 on a 0-16 scale). The correlation matrix has a striped band structure: the main diagonal, plus bands eight steps away, which are the pixels directly above and below. The mean absolute correlation between neighbouring pixels is 0.42, against 0.09 for pixels at least six steps apart. The strongest negative pair (p54 and p56, rho = -0.55) lies in the same row two columns apart: a stroke passes through one or the other, rarely both. Information is local, which favours a convolutional network over a model that treats the 64 pixels as unrelated inputs.

4.7 Shape Features for Confusable Digits

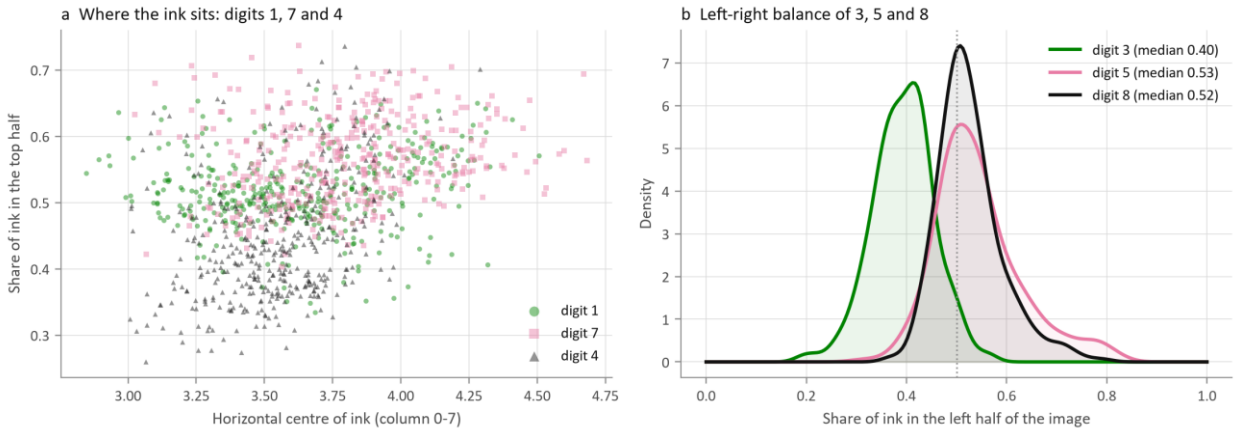


Fig. 4.6 Ink position for digits 1, 7 and 4, and the left-right ink balance of 3, 5 and 8

What it shows	Panel (a) plots, for every training 1, 7 and 4, the horizontal centre of its ink against the share of ink in the top half. Panel (b) compares the distribution of the left-half ink share for 3, 5 and 8.
Why it was created	Bivariate views of simple, explainable features show which pairs of digits are easy to separate and which need the richer representation of a neural network.
Insight obtained	Digit 4 sits low (median top share 0.43) and digit 7 sits high and to the right (top share 0.56, centre column 3.79), so these two separate well, while digit 1 overlaps both. In panel (b), 3 puts most of its ink on the right (median left share 0.40), whereas 5 and 8 are balanced (0.53 and 0.52) and overlap almost completely. A two-feature rule cannot split 5 from 8; the network has to learn finer stroke patterns.

4.8 Multivariate Structure: PCA and t-SNE

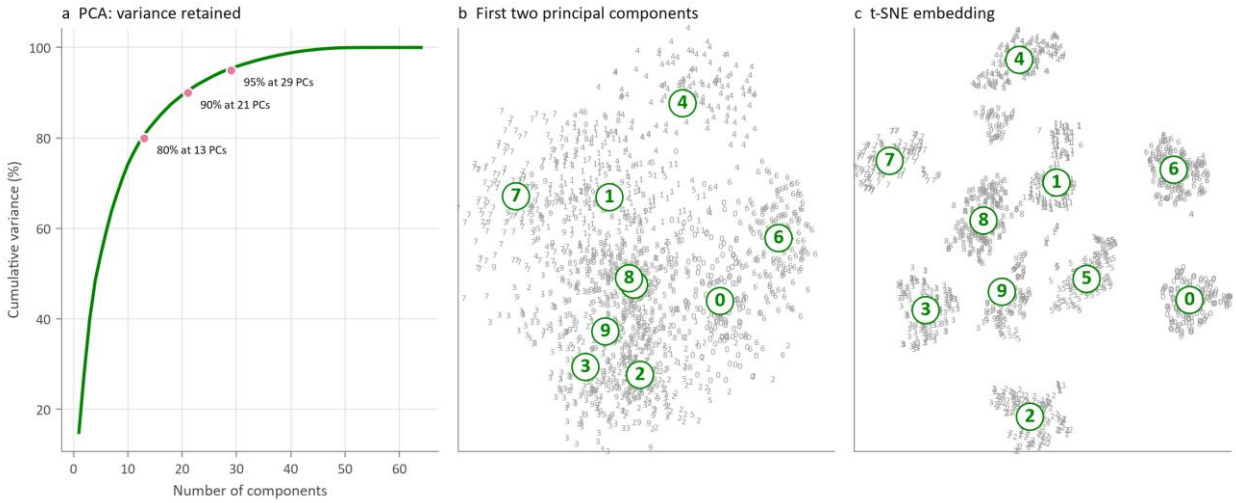


Fig. 4.7 Cumulative variance of the principal components, the first two components and a t-SNE map; each point is drawn as its digit

What it shows	Panel (a) is the cumulative variance explained by principal components. Panels (b) and (c) place 1,500 random training images by their first two components and by a t-SNE embedding, writing each image as its own digit and circling the class medians.
Why it was created	Multivariate methods show whether the 64-dimensional data contain natural groups. Writing each point as its digit avoids the ten-colour legend that no reader can decode.
Insight obtained	The variance is spread out: PC1 and PC2 explain only 14.9% and 13.4%, and 13, 21 and 29 components are needed for 80%, 90% and 95%. A linear projection therefore mixes the classes (panel b), apart from 0, 4, 6 and 7, which sit at the edges. t-SNE, which preserves local neighbourhoods, separates the ten digits into distinct islands: 98.6% of images share the label of most of their 10 nearest neighbours in the map. The classes are well separated, but along curved boundaries, which suits a non-linear model.

4.9 Unusual Handwriting

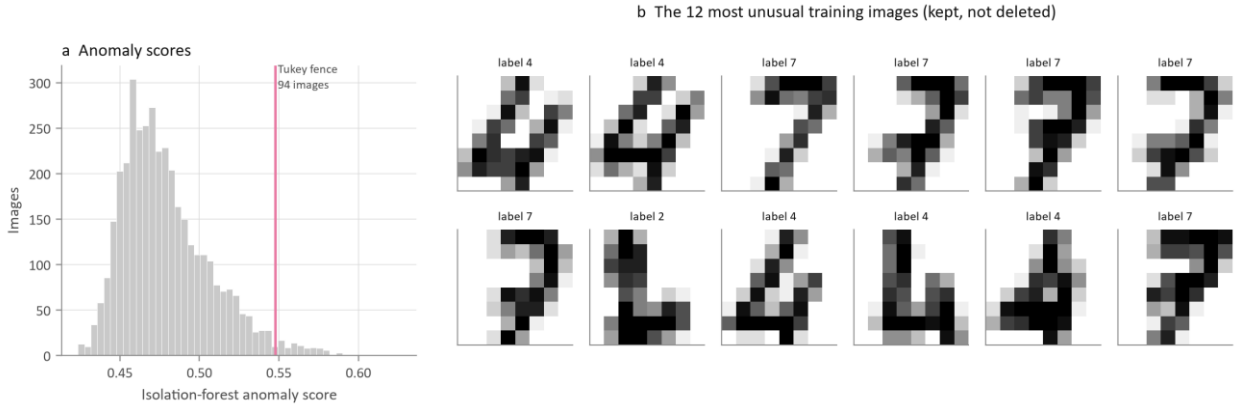


Fig. 4.8 Isolation-forest anomaly scores and the twelve most atypical training images

What it shows	The distribution of isolation-forest anomaly scores with Tukey's upper fence, and the twelve training images with the highest scores.
Why it was created	Outlier analysis for images must find unusual shapes rather than extreme numbers, and must show them so a person can judge whether they are errors.
Insight obtained	94 images (2.46%) exceed the fence. Among the 100 most unusual, 42 are 7s and 30 are 4s: continental 7s with a crossbar or a long tail, and 4s with an open top. All are legible and correctly labelled, so they were kept. They are a reminder that handwriting styles, not data errors, are the main source of difficulty.

4.10 Do the Two Groups of Writers Differ?

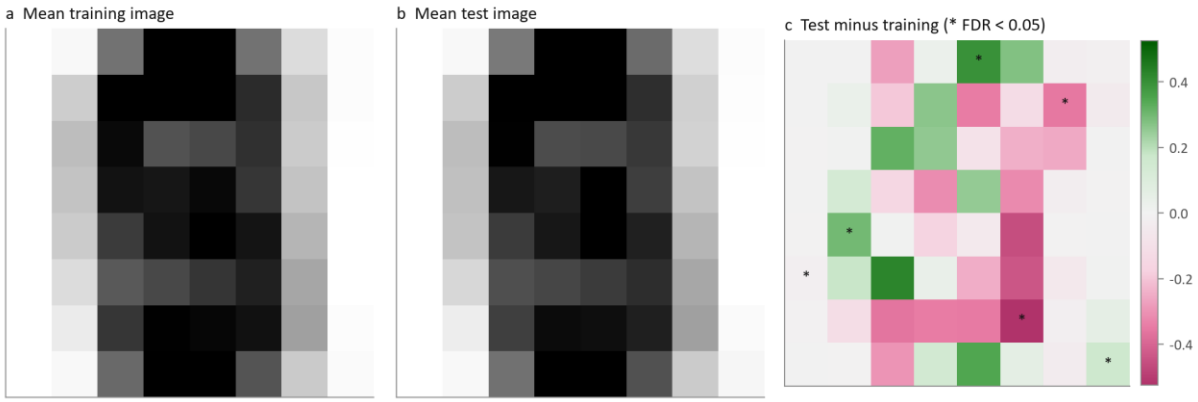


Fig. 4.9 Mean training and test images and their difference (stars: significant after FDR correction)

What it shows	The average image of all training digits, the average test digit, and their difference, with Welch t-tests for every pixel corrected for 64 comparisons (Benjamini-Hochberg).
Why it was created	Because the test writers are different people, any systematic difference between the groups would lower test accuracy. This chart measures that distribution shift directly.
Insight obtained	The shift is small: no pixel differs by more than 0.53 on the 0-16 scale, and only 6 of 64 pixels differ significantly. Mean ink is 315.1 for training and 312.6 for test writers. Individual writers vary far more than the two groups do, so a model that generalises across writers in validation should also do so on the test set.

4.11 Interact: the Digit Lab

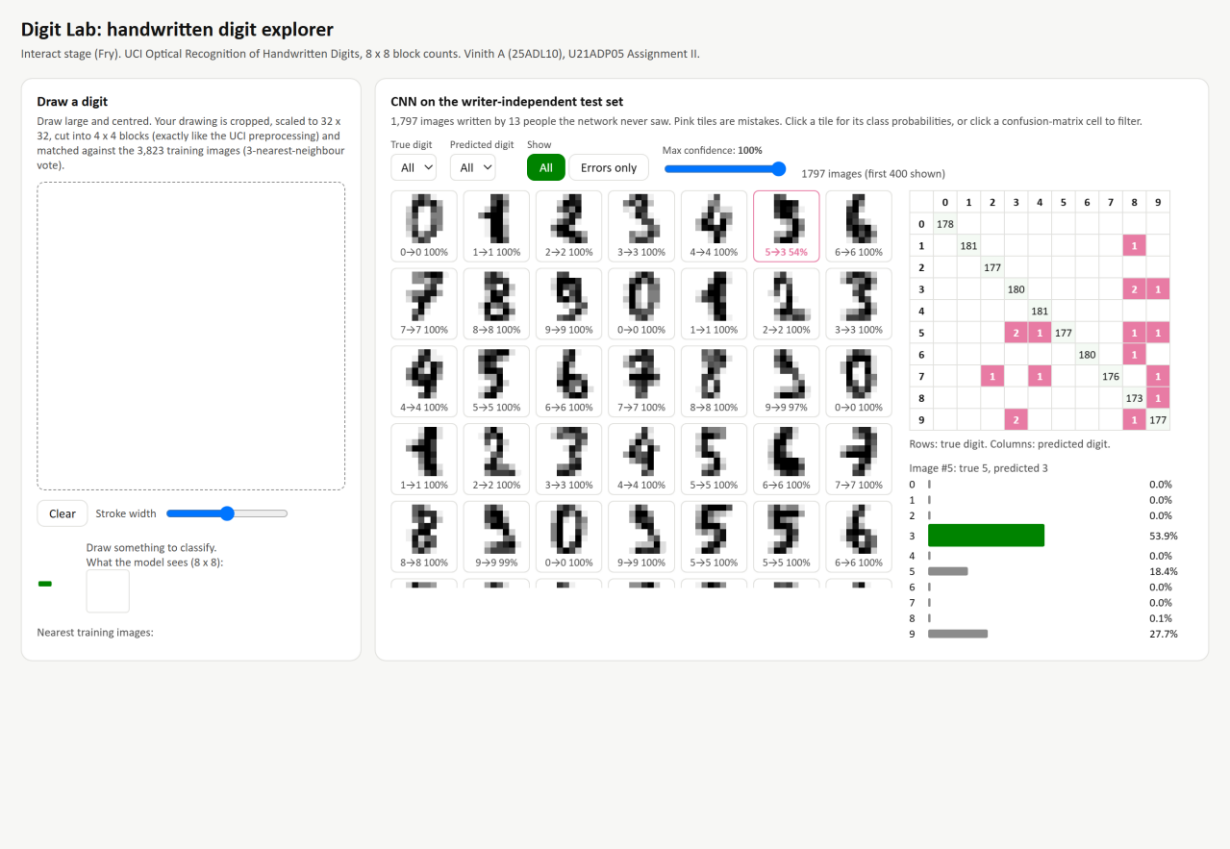


Fig. 4.10 The Digit Lab page (dashboard/index.html): draw-a-digit panel and test-set explorer

What it shows	A self-contained web page with two tools. On the left, a drawing pad converts the reader's own handwriting into 8 x 8 counts using the same block-counting rule and classifies it by nearest neighbours. On the right, a gallery of the 1,797 test images with the CNN's prediction and confidence, filters for true digit, predicted digit, errors only and maximum confidence, a clickable confusion matrix, and a probability bar chart for any selected image.
Why it was created	Interaction is Fry's final stage. Drawing a digit makes the preprocessing tangible, and filtering the predictions lets a reader inspect every mistake instead of trusting a single accuracy number.
Insight obtained	Clicking the off-diagonal cells shows that 15 of the 17 CNN errors involve 3, 5, 8 or 9, the same group whose prototypes overlapped in Section 4.3. Drawing digits shows how sensitive the 8 x 8 representation is to stroke thickness, which explains why augmentation with small shifts helps.

4.12 Summary of the Exploratory Findings

Table 4.3 – Summary of the statistical analysis

Question	**Method**	**Finding**
Balanced classes?	Chi-square	Yes; no difference between writer groups
Pixel-value law?	Binomial vs binomial mixture (EM)	U-shaped; three regimes: background, edge, core
Ink distribution?	Normal, skew-normal, Shapiro-Wilk, Kruskal-Wallis	Near-normal, slight right skew; digits differ only slightly
Where is the information?	Pixel SD, P(ink) maps	Inner six columns; corners empty
Redundancy?	Spearman matrix	Strong local correlation, weak long-range
Natural groups?	PCA, t-SNE	Linear mixing; non-linear separation (98.6% neighbour purity)
Outliers?	Isolation forest	2.5% unusual styles, mostly 7s and 4s; kept
Writer shift?	Welch t-tests with FDR	Small; 6 of 64 pixels

5 Deep Learning Model

5.1 Why a Convolutional Network

The exploration showed that information is local (Section 4.6), that a digit keeps its identity when it moves slightly, and that the classes are separated along curved boundaries (Section 4.8). A convolutional neural network [5] matches all three: small filters look at local neighbourhoods, the same filter is applied at every position, and stacked non-linear layers can draw curved decision boundaries. Each convolution computes

$$h_{i,j} = \mathrm{ReLU}(\mathrm{BN}(\Sigma_{u,v} w_{u,v} \cdot x_{i+u,\, j+v}))$$

over a 3 x 3 window, and the final softmax layer turns ten scores into probabilities that sum to one.

5.2 Architecture

Table 5.1 – Architecture of the selected CNN

Block	Layers	Output shape
Input	8 x 8 x 1 image scaled to [0, 1]; optional random shift (training only)	8 x 8 x 1
Convolution block 1	2 x [Conv 3 x 3 (32 filters, same padding) + BatchNorm + ReLU], MaxPool 2 x 2, Dropout	4 x 4 x 32
Convolution block 2	Conv 3 x 3 (64 filters) + BatchNorm + ReLU, MaxPool 2 x 2	2 x 2 x 64
Classifier	Flatten, Dense 64 (ReLU, the 'embedding'), Dropout, Dense 10 (softmax)	10
Total	45,546 trainable and non-trainable parameters	

5.3 Training Parameters

Table 5.2 – Training configuration

Setting	Value
Framework	TensorFlow 2.21.0 / Keras 3.15.1, seed 7, deterministic operations
Loss	Sparse categorical cross-entropy
Optimiser	Adam, learning rate 0.001
Batch size / maximum epochs	64 / 120
Early stopping	Validation loss, patience 12, best weights restored
Learning-rate schedule	Halve on a 5-epoch plateau of validation loss (minimum 1e-5)
Regularisation	Batch normalisation, dropout (tuned), random one-pixel shifts (tuned)

5.4 Hyperparameter Tuning

Table 5.3 – Grid search over filters, dropout and augmentation (selected: #6)

#	Filters	Dropout	Shift aug.	Epochs	Best epoch	Val. accuracy	Val. loss	Params
1	16-32	0.25	no	33	21	98.95%	0.0303	16,218
2	16-32	0.25	yes	37	25	98.08%	0.0476	16,218
3	16-32	0.5	no	34	22	98.95%	0.0343	16,218
4	16-32	0.5	yes	37	25	98.08%	0.0549	16,218
5	32-64	0.25	no	31	19	99.13%	0.0272	45,546
6	32-64	0.25	yes	49	37	99.30%	0.0143	45,546
7	32-64	0.5	no	42	30	99.13%	0.0290	45,546
8	32-64	0.5	yes	39	27	99.30%	0.0297	45,546

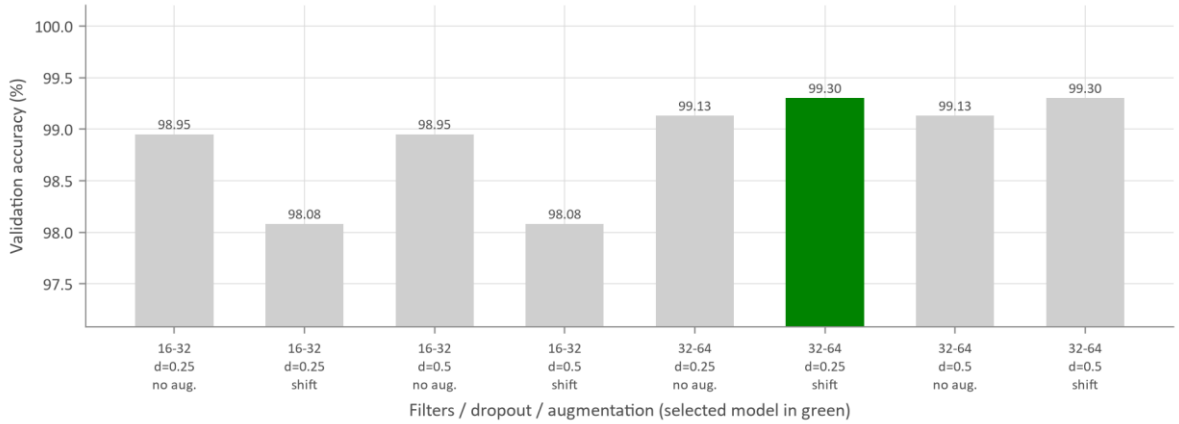


Fig. 5.1 Validation accuracy of the eight configurations

What it shows	Validation accuracy of each of the eight combinations of filter count, dropout rate and augmentation, with the selected configuration highlighted.
Why it was created	Basic tuning shows which design choices matter. Selection used validation accuracy, with validation loss as the tie-breaker.
Insight obtained	Capacity matters most: the wider 32-64 networks reach 99.13-99.30%, against 98.08-98.95% for the narrow ones. Shift augmentation helps the wide network but hurts the narrow one, which lacks the capacity to learn shifted variants. Dropout makes little difference. The selected model (#6) ties for the best accuracy (99.30%) and has the lowest validation loss (0.0143), meaning its correct answers are also the most confident.

5.5 Feature Analysis: What the Network Looks At

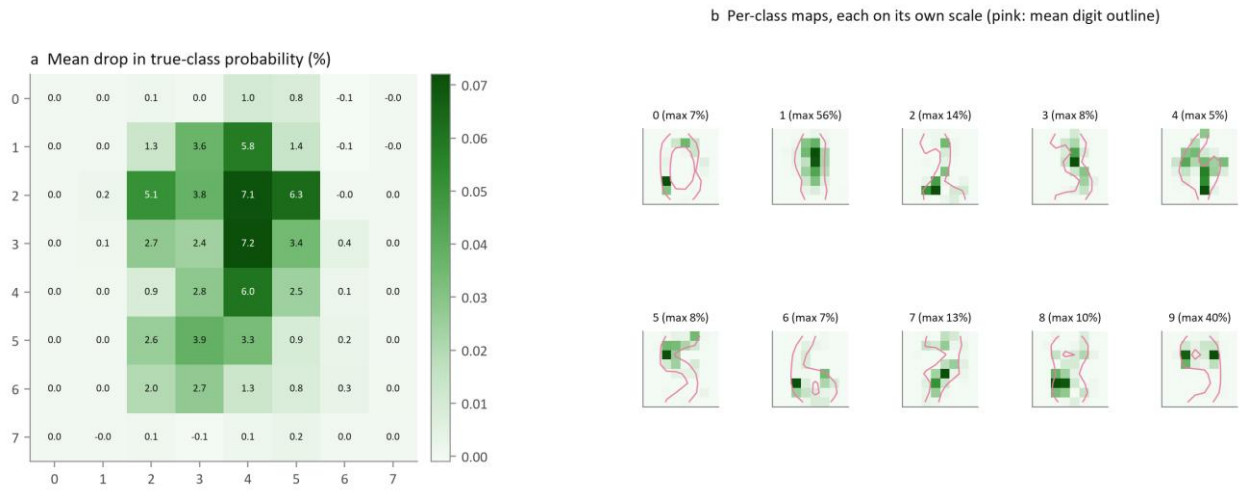


Fig. 5.2 Occlusion sensitivity: drop in the probability of the true digit when one pixel is erased

What it shows	For each of the 64 positions, that pixel was set to zero in every validation image and the average fall in the probability of the correct digit was recorded, overall (a) and for each digit on its own scale (b), with the outline of the mean digit for reference.
Why it was created	Occlusion analysis [8] is a model-agnostic form of feature importance for images. It shows whether the network relies on sensible, digit-specific evidence.
Insight obtained	The network ignores the empty borders (mean drop 0.072%) and relies on the centre of the grid (mean drop 3.81%), exactly where Section 4.6 found the variability. Erasing one pixel rarely matters, because the evidence for a digit is spread over many pixels, which makes the model robust. The per-class maps are digit-specific: for a 1, erasing the top of the central bar (row 2, column 4) lowers the correct probability by 56%, and for a 9 the top-right pixel where the loop closes (row 2, column 5) costs 40%. That is exactly the stroke that separates a 9 from a 3 or 5. For the other digits no single pixel matters much.

5.6 Baselines

Two classical models were trained on the same split for comparison: k-nearest neighbours [13] with k chosen on validation data (k = 3, validation accuracy 98.78%) and multinomial logistic regression (validation accuracy 95.99%). k-NN is a demanding benchmark here, because nearby writers produce near-identical count patterns.

6 Result Visualization and Interpretation

All results in this section are on the 1,797 test images written by the 13 people who contributed nothing to training or validation.

6.1 Loss and Accuracy versus Epoch

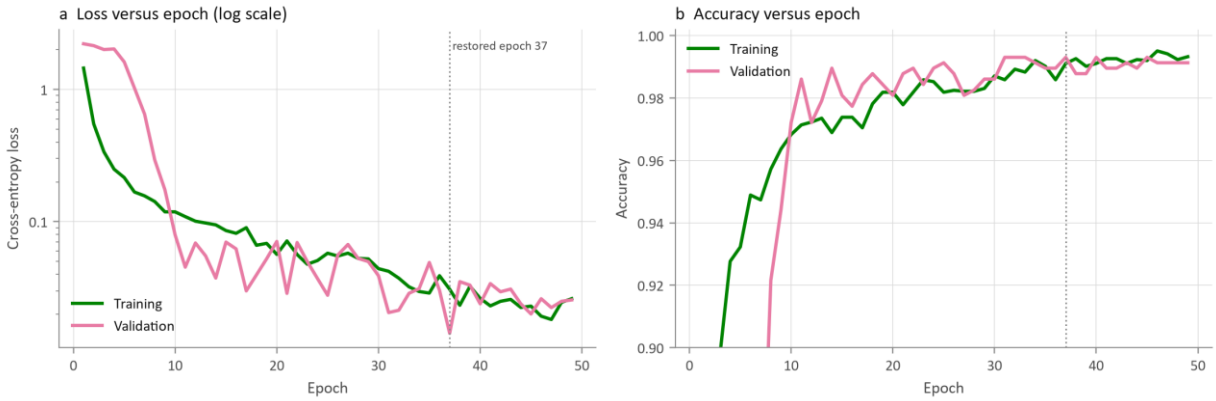


Fig. 6.1 Training and validation loss (log scale) and accuracy for each epoch

What it shows	Cross-entropy loss and accuracy on the training and validation sets after every epoch of the selected model, with the restored epoch marked.
Why it was created	Learning curves show whether the model converged and whether it over-fitted.
Insight obtained	Validation accuracy climbs above 98% within the first dozen epochs; training stopped at epoch 49 and the weights from epoch 37 (lowest validation loss, 0.0143) were restored. At that point training and validation accuracy are 99.11% and 99.30%. The validation curves stay at or above the training curves because dropout and random shifts are active only during training, so the gap is a sign of regularisation, not over-fitting. The early validation loss is high because batch-normalisation statistics settle only after a few epochs.

6.2 Confusion Matrix

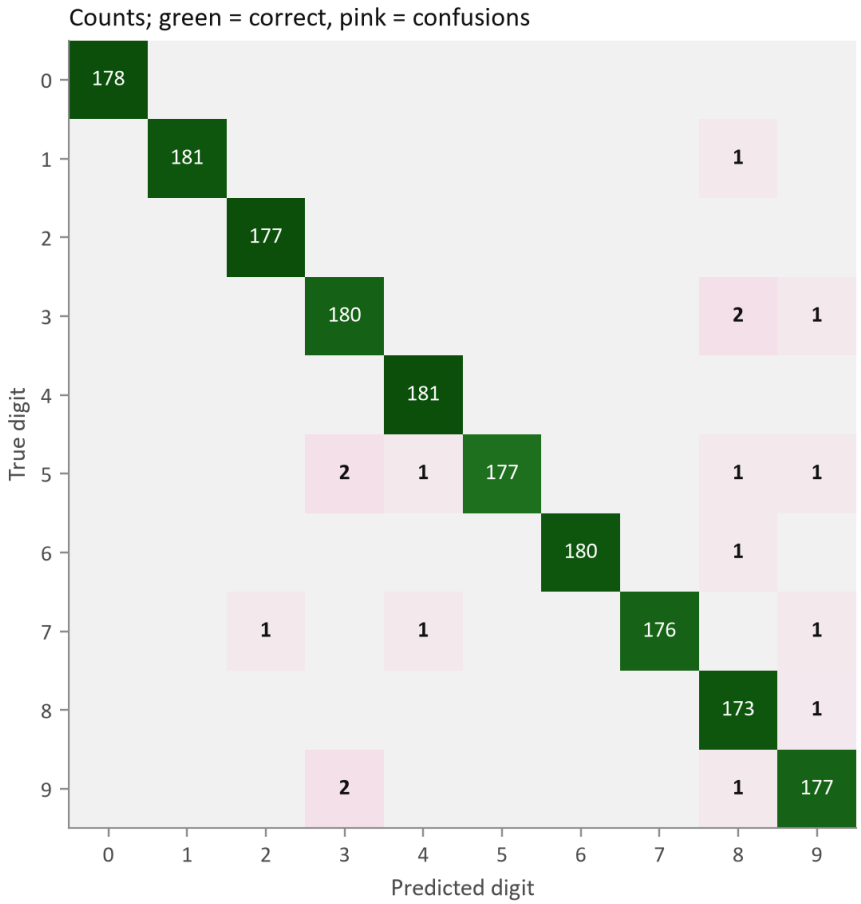


Fig. 6.2 Confusion matrix of the CNN on the writer-independent test set

What it shows	Counts of test images by true digit (rows) and predicted digit (columns). The diagonal holds correct answers and every pink cell is a type of mistake.
Why it was created	A ten-class confusion matrix shows which digits are mistaken for which, information that accuracy hides.
Insight obtained	Only 17 of 1,797 images are misclassified, spread over many cells with at most 2 errors each. The repeated confusions are 9 read as 3, 5 read as 3, 3 read as 8: digits sharing the right-hand curve seen in the prototypes. Digit 0 was recognised without a single mistake.

6.3 Precision, Recall and F1 by Digit

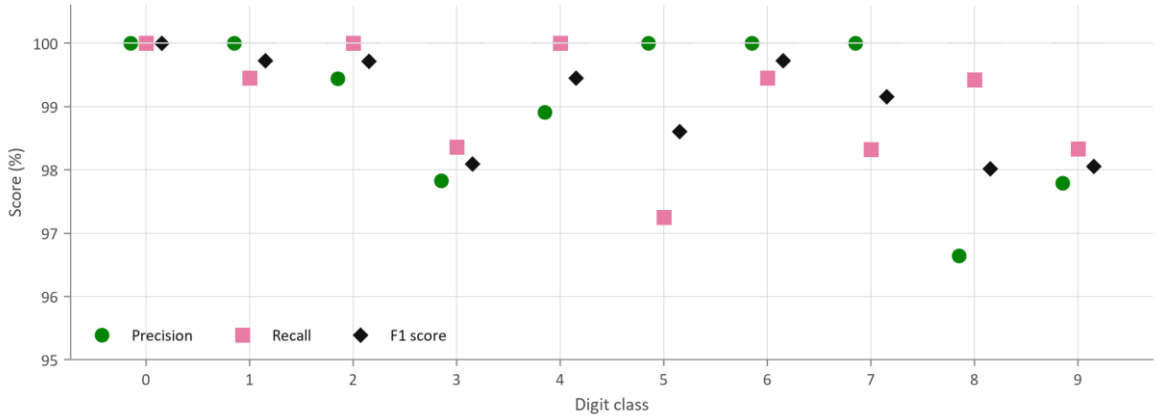


Fig. 6.3 Precision, recall and F1 score for each digit

What it shows	For each digit, precision (how often a prediction of that digit is right), recall (how many of that digit are found) and their harmonic mean F1.
Why it was created	Per-class scores show whether a good average hides a weak digit.
Insight obtained	Every digit scores at least 96.7%. The lowest recall is for 5 (97.25%) and the lowest precision for 8 (96.65%), because unusual digits tend to be pulled towards 8, whose loops overlap with many other shapes. The macro-averaged F1 is 0.9905.

6.4 ROC Curves and AUC

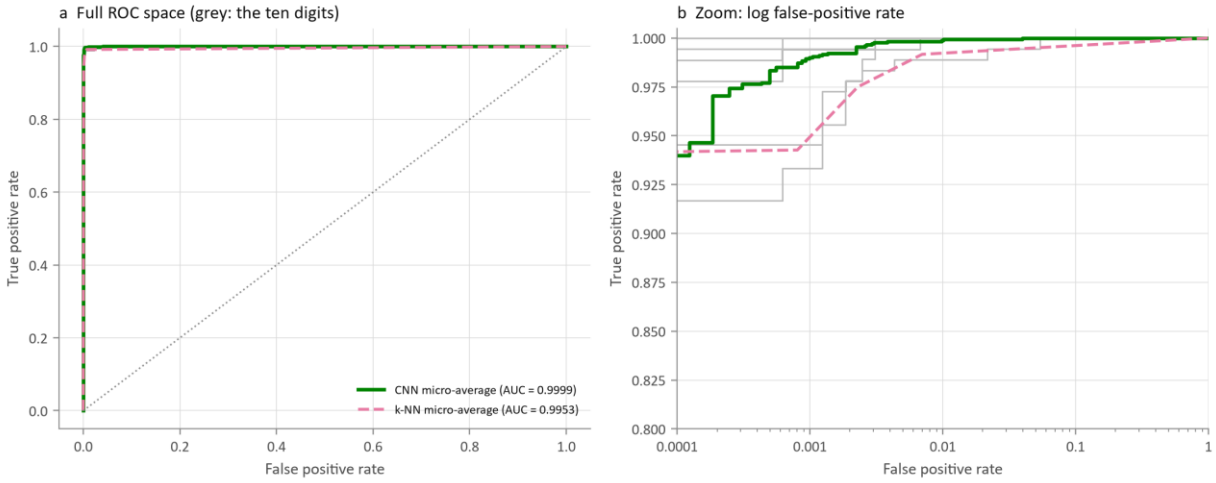


Fig. 6.4 One-versus-rest ROC curves of the ten digits with micro-averages for the CNN and k-NN

What it shows	One-versus-rest ROC curves for every digit (grey) with the micro-averaged curve of the CNN and of k-NN, on a linear axis (a) and on a logarithmic false-positive axis that zooms in on the region of interest (b).
Why it was created	ROC analysis evaluates the ranking quality of the predicted probabilities independently of any threshold. For near-perfect models the logarithmic axis is needed to see differences.
Insight obtained	The CNN's micro-averaged AUC is 0.9999 and its macro-averaged AUC 0.9999; the lowest per-digit AUC is 0.9995. On the linear axis the curves hug the corner. The zoomed view shows the real gap: at very low false-positive rates the CNN still finds clearly more true positives than k-NN, whose macro AUC is 0.9952.

6.5 The Mistakes

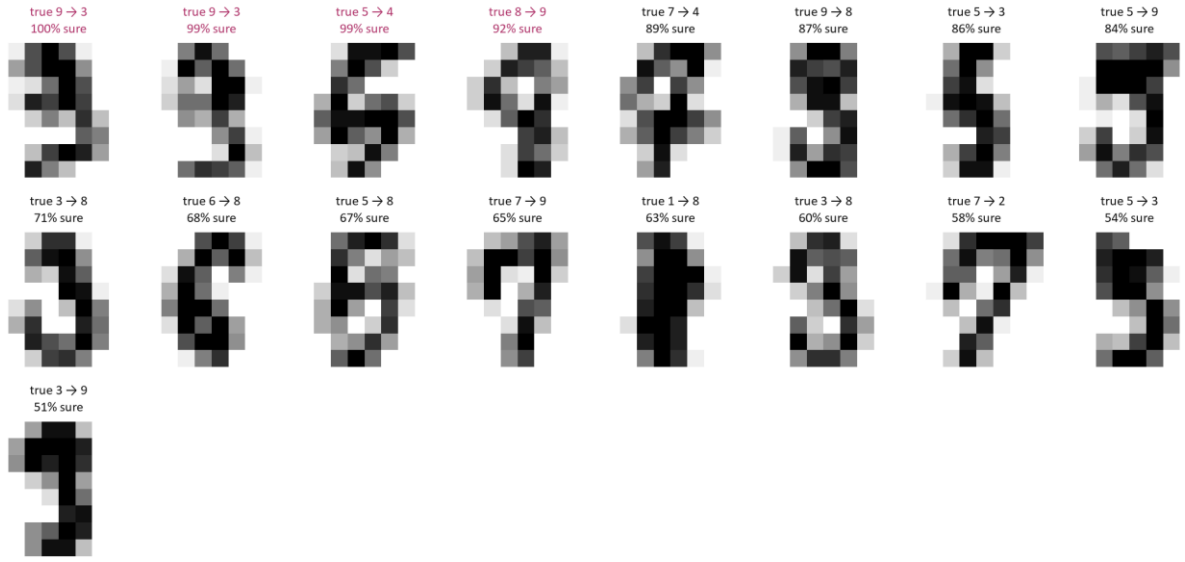


Fig. 6.5 All misclassified test images, sorted by the network's confidence

What it shows	Every one of the 17 test errors with its true label, predicted label and confidence, most confident first.
Why it was created	Looking at the errors themselves is the most honest evaluation: it shows whether mistakes are understandable or absurd.
Insight obtained	Eleven of the 17 errors are confusions within the group 3, 5, 8 and 9, and fifteen involve at least one of those digits, so the mistakes concentrate exactly where the exploratory analysis predicted. 4 errors were made with more than 90% confidence (two 9s read as 3, a 5 read as 4 and an 8 read as 9); these would slip through an automatic confidence check.

6.6 Confidence, Calibration and the Error Distribution

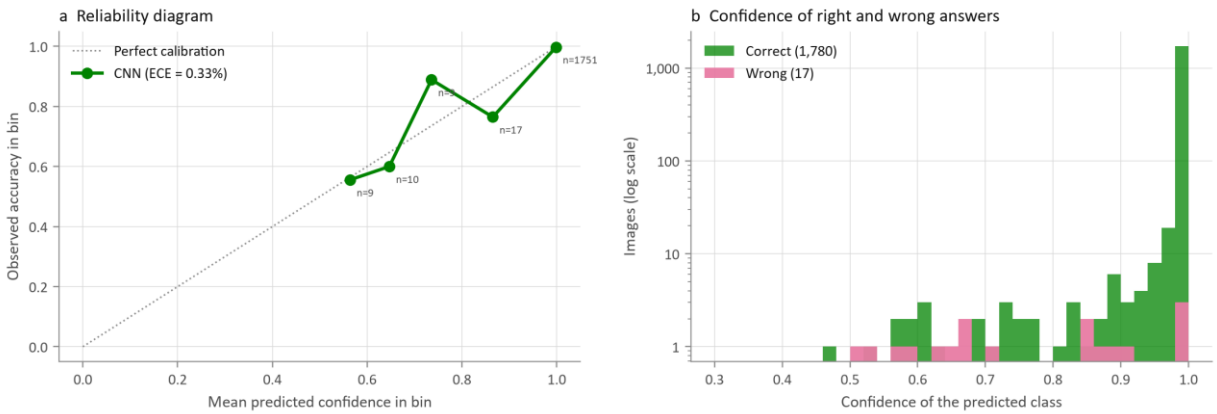


Fig. 6.6 Reliability diagram and confidence of correct and incorrect predictions

What it shows	Panel (a) groups test images by predicted confidence and compares it with the accuracy actually achieved; panel (b) compares the confidence of correct and incorrect answers on a log count scale.
Why it was created	A model used to process real forms should know when it is unsure. Calibration [11] measures whether a stated 90% confidence really means 90% correct.
Insight obtained	The expected calibration error is only 0.33%. Correct answers have a median confidence of 1.000 and wrong answers 0.71, so confidence is informative. In practice, accepting automatically only predictions above 90% confidence covers 97.4% of forms with 99.77% accuracy, and sends the rest to a person.

6.7 What the Network Learned

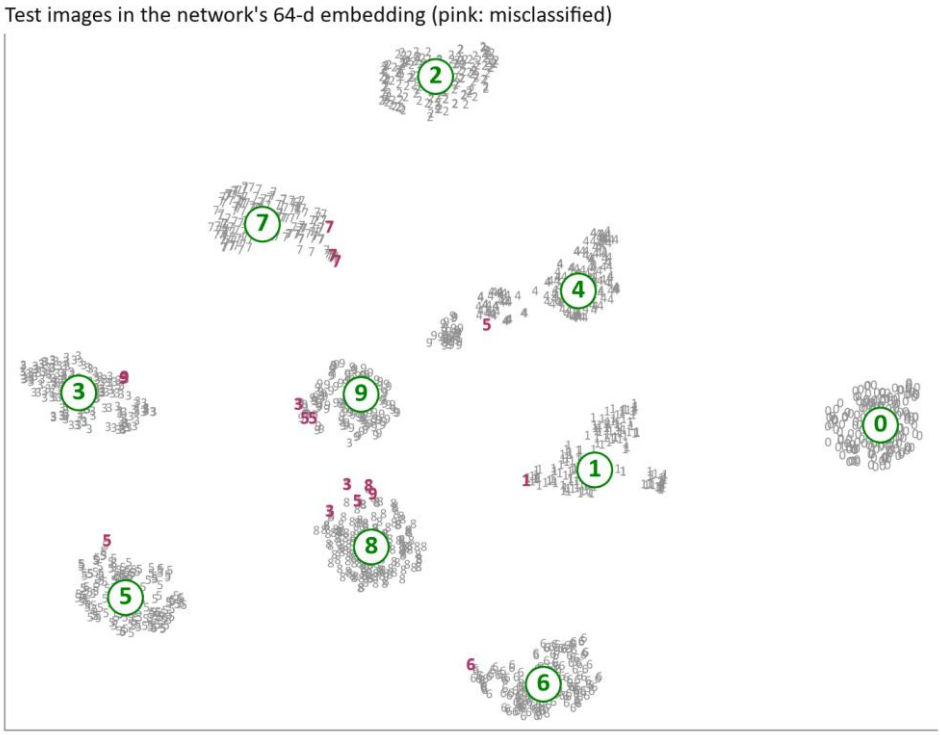


Fig. 6.7 t-SNE map of the network's 64-dimensional embedding for the test images (pink: errors)

What it shows	The test images placed by t-SNE using the network's penultimate layer instead of the raw pixels, with misclassified images in pink.
Why it was created	Comparing this map with the raw-pixel map (Fig. 4.7) shows what the network learned beyond the data's own structure.
Insight obtained	The learned representation forms ten compact, well-separated islands, tighter than those of the raw pixels. Almost all errors lie between islands, on the borders between 3, 5, 8 and 9, where the image is genuinely ambiguous. Very few errors sit deep inside a wrong island.

6.8 Comparison with the Baselines

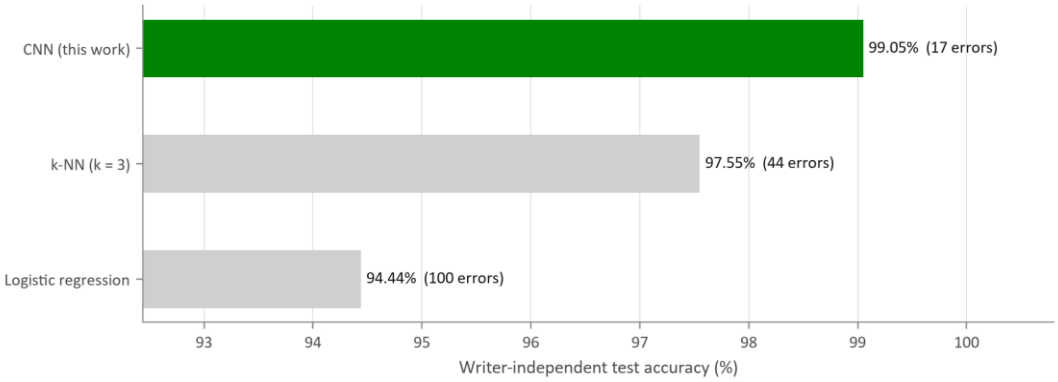


Fig. 6.8 Test accuracy of the CNN and the two baselines

Table 6.1 – Test-set performance

Metric	CNN	k-NN (k = 3)	Logistic regression
Accuracy	99.05%	97.55%	94.44%
Macro F1	0.9905	0.9756	0.9446
Cohen's kappa	0.9895	0.9728	0.9382
Log loss	0.0304	0.1660	0.1728
Macro AUC	0.9999	0.9952	0.9984
Errors (of 1,797)	17	44	100

What it shows	Test accuracy and error counts for the three models, with further metrics in the table.
Why it was created	A deep model is only worth its complexity if it clearly beats simpler methods on the same data.
Insight obtained	The CNN makes 17 errors against 44 for k-NN and 100 for logistic regression. The 95% Wilson interval for its accuracy is 98.49%-99.41%. On the same images, the CNN is right where k-NN is wrong 37 times and the reverse happens 10 times; McNemar's test [12] gives p = 0.00010, so the improvement is statistically significant. Logistic regression has a good AUC but a much higher error rate, because a linear boundary cannot follow the curved class borders seen in Section 4.8.

7 Conclusion and Future Scope

7.1 What Was Learned from the Data

- The dataset is clean, balanced and leak-free, and its 64 features were reproduced exactly from the original bitmaps.
- Pixel counts follow a U-shaped law that a single binomial cannot describe; a three-component binomial mixture (background 54%, edge 23%, stroke core 24%) explains it physically.
- Total ink is nearly normal and similar for all digits, because preprocessing normalises size; shape, not quantity, identifies a digit.
- Information is local and concentrated in the inner columns, and pixels correlate strongly only with their neighbours.
- The classes are separated along curved boundaries (clear in t-SNE, mixed in PCA), and 3, 5, 8 and 9 are the hard group.
- Unusual samples are genuine handwriting styles, and the two groups of writers differ only slightly.

7.2 Model Performance

The tuned CNN (32-64 filters, dropout 0.25, shift augmentation, 45,546 parameters) read the handwriting of 13 unseen writers with 99.05% accuracy (17 errors in 1,797), a macro F1 of 0.9905 and a micro-averaged AUC of 0.9999. It was significantly better than k-NN (97.55%) and logistic regression (94.44%), its confidence was well calibrated (ECE 0.33%), and occlusion analysis showed that it relies on the central, information-rich region identified by the exploratory analysis.

7.3 Challenges Faced

- The original bitmaps were stored in the obsolete .Z format, which needed a custom, verified decoder.
- Ten classes exceed what colour can distinguish; charts use digit glyphs as markers and colour only for two or three series.
- Standard outlier rules do not suit bounded image data; an isolation forest and visual inspection were used instead.
- With accuracy near 99%, differences between models are small, so a paired significance test and log-scale ROC plots were needed to show them.
- The small 8 x 8 input limits how many pooling layers a CNN can use without losing all spatial detail.

7.4 Future Scope

- Train directly on the 32 x 32 bitmaps, which keep stroke detail lost in the 4 x 4 counting.
- Use stronger augmentation (small rotations, elastic distortions [7]) to cover the unusual 4s and 7s found in Section 4.9.

- Add writer-adaptive fine-tuning: a few labelled samples from a new writer could correct their personal style.
- Route low-confidence predictions to a human reviewer and measure the cost saving, using the calibration results.
- Extend from single digits to whole numbers on forms (segmentation plus recognition), the real task in banks and post offices.

References

[1] E. Alpaydin and C. Kaynak, “Optical Recognition of Handwritten Digits,” UCI Machine Learning Repository, 1998. [Online]. Available: https://doi.org/10.24432/C50P49

[2] C. Kaynak, “Methods of combining multiple classifiers and their applications to handwritten digit recognition,” M.Sc. thesis, Inst. Graduate Studies in Science and Engineering, Bogazici Univ., Istanbul, Turkey, 1995.

[3] E. Alpaydin and C. Kaynak, “Cascading classifiers,” *Kybernetika*, vol. 34, no. 4, pp. 369–374, 1998.

[4] M. D. Garris, J. L. Blue, G. T. Candela, D. L. Dimmick, J. Geist, P. J. Grother, S. A. Janet, and C. L. Wilson, “NIST form-based handprint recognition system,” Nat. Inst. Standards Technol., Gaithersburg, MD, USA, NISTIR 5469, 1994.

[5] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.

[6] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in *Proc. 32nd Int. Conf. Machine Learning (ICML)*, 2015, pp. 448–456.

[7] P. Y. Simard, D. Steinkraus, and J. C. Platt, “Best practices for convolutional neural networks applied to visual document analysis,” in *Proc. 7th Int. Conf. Document Analysis and Recognition (ICDAR)*, 2003, pp. 958–963.

[8] M. D. Zeiler and R. Fergus, “Visualizing and understanding convolutional networks,” in *Computer Vision – ECCV 2014*, LNCS, vol. 8689. Cham, Switzerland: Springer, 2014, pp. 818–833.

[9] L. van der Maaten and G. Hinton, “Visualizing data using t-SNE,” *J. Machine Learning Research*, vol. 9, pp. 2579–2605, 2008.

[10] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation forest,” in *Proc. 8th IEEE Int. Conf. Data Mining (ICDM)*, 2008, pp. 413–422.

[11] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” in *Proc. 34th Int. Conf. Machine Learning (ICML)*, 2017, pp. 1321–1330.

[12] Q. McNemar, “Note on the sampling error of the difference between correlated proportions or percentages,” *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.

[13] T. M. Cover and P. E. Hart, “Nearest neighbor pattern classification,” *IEEE Trans. Inf. Theory*, vol. 13, no. 1, pp. 21–27, Jan. 1967.

[14] B. Fry, *Visualizing Data*. Sebastopol, CA, USA: O’Reilly Media, 2008.

[15] F. Chollet *et al.*, “Keras,” 2015. [Online]. Available: https://keras.io

Appendix A: Repository and Reproduction

The project is published at **https://github.com/25adl10-arch/handwritten-digits-cnn-eda**. Each pipeline step is a separate script so that it can be run and read on its own; the notebook runs all of them in order with narration.

```
handwritten-digits-cnn-eda/
|-- README.md                      summary, results and instructions
|-- requirements.txt
|-- data/optdigits.zip             original UCI archive (unmodified)
|-- src/common.py                  acquire + parse: zip reader, LZW decoder, plotting style
|-- src/step2_explore.py           filter + mine: audit, distributions, PCA, t-SNE, outliers
|-- src/step3_train_cnn.py         CNN, hyper-parameter grid, occlusion, baselines
|-- src/step4_evaluate.py          test-set evaluation and result figures
|-- src/step5_dashboard.py         builds the Digit Lab page
|-- src/dashboard_template.html
|-- notebooks/digit_case_study.ipynb   executed walkthrough
|-- dashboard/index.html           interactive Digit Lab (works offline)
|-- figures/                       all figures in this report
|-- results/                       eda.json, model.json, evaluation.json, cnn_best.keras
`-- report/                        this report (PDF and DOCX)
```

To reproduce every number (about seven minutes on a laptop CPU):

```
pip install -r requirements.txt
jupyter nbconvert --to notebook --execute --inplace notebooks/digit_case_study.ipynb
```

Appendix B: Source Code (Key Sections)

Extracts from the scripts in `src/`; plotting code is omitted for brevity.

B.1 Data loading and the LZW decoder (common.py)

```
def load_tables():
    """The two preprocessed 8 x 8 tables: 64 integer counts (0-16) + class label."""
    with zipfile.ZipFile(ZIP) as z:
        tra = pd.read_csv(io.BytesIO(z.read("optdigits.tra")), header=None, names=PIXELS + ["digit"])
        tes = pd.read_csv(io.BytesIO(z.read("optdigits.tes")), header=None, names=PIXELS + ["digit"])
    return tra, tes


def unlzw(data: bytes) -> bytes:
    """Decoder for the Unix `compress` (.Z, LZW) format used by the original UCI bitmap files."""
    if data[:2] != b"\x1f\x9d":
        raise ValueError("not a .Z file")
    maxbits, block_mode = data[2] & 0x1F, bool(data[2] & 0x80)
    out = bytearray()
    table = [bytes([i]) for i in range(256)]
    if block_mode:
        table.append(b"")                       # code 256 = CLEAR
    n_bits, pos = 9, 3 * 8                      # bit position after the 3-byte header
    group_start = pos
    prev = None
    total_bits = len(data) * 8
    while pos + n_bits <= total_bits:
        byte_i, bit_i = divmod(pos, 8)
        chunk = int.from_bytes(data[byte_i:byte_i + 4].ljust(4, b"\0"), "little")
        code = (chunk >> bit_i) & ((1 << n_bits) - 1)
        pos += n_bits
        if block_mode and code == 256:
            # codes are written in groups of n_bits bytes; skip to the end of the current group
            used = pos - group_start
            pos += (-used) % (n_bits * 8)
            table = table[:257]
            n_bits, prev = 9, None
            group_start = pos
            continue
        if code < len(table):
            entry = table[code]
            if prev is not None:
                table.append(prev + entry[:1])
        elif code == len(table) and prev is not None:
            entry = prev + prev[:1]
            table.append(entry)
        else:
            raise ValueError("corrupt LZW stream")
        out += entry
        prev = entry
        if len(table) >= (1 << n_bits) and n_bits < maxbits:
            used = pos - group_start
            pos += (-used) % (n_bits * 8)
            n_bits += 1
            group_start = pos
    return bytes(out)


def load_bitmaps(member="optdigits-orig.windep.Z"):
    """Original 32 x 32 binary bitmaps and labels from one of the .Z files."""
    with zipfile.ZipFile(ZIP) as z:
        text = unlzw(z.read(member)).decode("ascii")
    lines = text.splitlines()
    start = next(i for i, ln in enumerate(lines) if len(ln.strip()) == 32 and set(ln.strip()) <= {"0", "1"})
    imgs, labels, i = [], [], start
    while i + 32 < len(lines):
        block = lines[i:i + 32]
        if not all(len(b.strip()) == 32 for b in block):
            break
        imgs.append(np.array([[int(ch) for ch in b.strip()] for b in block], dtype=np.uint8))
        labels.append(int(lines[i + 32].strip()))
        i += 33
    return np.stack(imgs), np.array(labels)
```

B.2 Data-quality audit and feature verification (step2_explore.py)

```
out["missing_cells"] = int(tra.isna().sum().sum() + tes.isna().sum().sum())
out["out_of_range"] = int(((tra[C.PIXELS] < 0) | (tra[C.PIXELS] > 16)).sum().sum()
                          + ((tes[C.PIXELS] < 0) | (tes[C.PIXELS] > 16)).sum().sum())
out["dup_rows_train"] = int(tra.duplicated().sum())
out["dup_rows_test"] = int(tes.duplicated().sum())
cross = pd.merge(tra[C.PIXELS].drop_duplicates(), tes[C.PIXELS].drop_duplicates(), how="inner")
out["identical_images_train_and_test"] = int(len(cross))
dup_feats = tra[C.PIXELS].duplicated()
conflicts = tra[tra[C.PIXELS].duplicated(keep=False)].groupby(C.PIXELS)["digit"].nunique()
out["dup_images_with_conflicting_labels"] = int((conflicts > 1).sum())
var = tra[C.PIXELS].var()
out["zero_variance_pixels"] = [p for p in C.PIXELS if var[p] == 0]
out["near_constant_pixels"] = [p for p in C.PIXELS if (tra[p] == 0).mean() > 0.99]
both = pd.concat([tra, tes])
out["pixels_zero_pct"] = round(100 * float((both[C.PIXELS] == 0).to_numpy().mean()), 2)
out["pixels_sixteen_pct"] = round(100 * float((both[C.PIXELS] == 16).to_numpy().mean()), 2)

# parse check: the 8 x 8 test table equals 4 x 4 block counts of the original 32 x 32 bitmaps
B, yb = C.load_bitmaps("optdigits-orig.windep.Z")
blocks = B.reshape(-1, 8, 4, 8, 4).sum(axis=(2, 4))
out["bitmap_to_table_exact_match_pct"] = round(100 * float(np.mean(np.all(blocks == Xte, axis=(1, 2)) & (yb ==
yte))), 2)
```

B.3 EM algorithm for the binomial mixture (step2_explore.py)

```
def binom_mix_em(v, k=3, iters=300):
    """EM for a k-component binomial mixture on counts 0..16 (ink regimes: background, edge, stroke core)."""
    cnt = np.bincount(v, minlength=n + 1)
    support = np.arange(n + 1)
    w = np.full(k, 1 / k)
    p = np.linspace(0.02, 0.95, k)
    for _ in range(iters):
        like = np.array([w[j] * stats.binom.pmf(support, n, p[j]) for j in range(k)])     # k x 17
        resp = like / like.sum(0, keepdims=True)
        wk = (resp * cnt).sum(1)
        w = wk / wk.sum()
        p = (resp * cnt * support).sum(1) / (n * wk)
    ll = float((cnt * np.log(like.sum(0) + 1e-300)).sum())
    return w, p, ll
```

B.4 Split, CNN definition and hyper-parameter search (step3_train_cnn.py)

```
tra, tes = C.load_tables()
X = C.to_images(tra)[..., None] / 16.0           # scale counts 0-16 to 0-1, add channel axis
y = tra["digit"].to_numpy()
Xte = C.to_images(tes)[..., None] / 16.0
yte = tes["digit"].to_numpy()
Xtr, Xva, ytr, yva = train_test_split(X, y, test_size=0.15, stratify=y, random_state=C.SEED)
C.dump("model", {"split": {"train": int(len(Xtr)), "validation": int(len(Xva)), "test": int(len(Xte))}})


def build(filters=(32, 64), dropout=0.25, augment=False, lr=1e-3):
    inp = keras.Input((8, 8, 1), name="image_8x8")
    x = inp
    if augment:                                  # shift by up to one pixel (1/8 of the image)
        x = layers.RandomTranslation(0.125, 0.125, fill_mode="constant", name="shift_augment")(x)
    for k in range(2):
        x = layers.Conv2D(filters[0], 3, padding="same", use_bias=False, name=f"conv1_{k + 1}")(x)
        x = layers.BatchNormalization(name=f"bn1_{k + 1}")(x)
        x = layers.ReLU(name=f"relu1_{k + 1}")(x)
    x = layers.MaxPooling2D(2, name="pool1")(x)                      # 8x8 -> 4x4
    x = layers.Dropout(dropout / 2, name="drop1")(x)
    x = layers.Conv2D(filters[1], 3, padding="same", use_bias=False, name="conv2")(x)
    x = layers.BatchNormalization(name="bn2")(x)
    x = layers.ReLU(name="relu2")(x)
    x = layers.MaxPooling2D(2, name="pool2")(x)                      # 4x4 -> 2x2
    x = layers.Flatten(name="flatten")(x)
    x = layers.Dense(64, activation="relu", name="embedding")(x)
    x = layers.Dropout(dropout, name="drop2")(x)
    out = layers.Dense(10, activation="softmax", name="digit_probabilities")(x)
    m = keras.Model(inp, out, name="digit_cnn")
    m.compile(optimizer=keras.optimizers.Adam(lr), loss="sparse_categorical_crossentropy", metrics=["accuracy"])
```

```
    return m


def fit(model, epochs):
    cb = [keras.callbacks.EarlyStopping(monitor="val_loss", patience=12, restore_best_weights=True),
          keras.callbacks.ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=5, min_lr=1e-5)]
    return model.fit(Xtr, ytr, validation_data=(Xva, yva), epochs=epochs, batch_size=64, callbacks=cb, verbose=0)


# ---------------------------------------------------------------- basic hyper-parameter search
grid = [dict(filters=f, dropout=d, augment=a) for f in [(16, 32), (32, 64)] for d in (0.25, 0.5) for a in (False,
True)]
if FAST:
    grid = grid[:2]
EPOCHS = 15 if FAST else 120
rows, best = [], None
t0 = time.time()
for i, cfg in enumerate(grid, 1):
    keras.utils.set_random_seed(C.SEED)
    m = build(**cfg)
    h = fit(m, EPOCHS)
    va_loss, va_acc = m.evaluate(Xva, yva, verbose=0)
    row = {"config": i, "filters": f"{cfg['filters'][0]}-{cfg['filters'][1]}", "dropout": cfg["dropout"],
           "augment": cfg["augment"], "epochs": len(h.history["loss"]),
           "best_epoch": int(np.argmin(h.history["val_loss"]) + 1), "val_acc": round(float(va_acc), 4),
           "val_loss": round(float(va_loss), 4), "params": int(m.count_params())}
    rows.append(row)
    print(row)
    key = (row["val_acc"], -row["val_loss"])
    if best is None or key > best[0]:
        best = (key, row, cfg, m, h.history)
_, brow, bcfg, model, hist = best
model.save(C.RES / "cnn_best.keras")
C.dump("model", {"tuning": rows, "tuning_seconds": round(time.time() - t0, 1), "best": brow,
                 "history": {k: [round(float(v), 5) for v in vals] for k, vals in hist.items()}})
model.summary(print_fn=lambda s, **kw: open(C.RES / "cnn_summary.txt", "a", encoding="utf-8").write(s + "\n")
              if s.strip() else None)

fig, ax = plt.subplots(figsize=(9, 3.4))
labels = [f"{r['filters']}\nd={r['dropout']}\n{'shift' if r['augment'] else 'no aug.'}" for r in rows]
cols = [C.GREEN if r["config"] == brow["config"] else "#cfcfcf" for r in rows]
ax.bar(range(len(rows)), [100 * r["val_acc"] for r in rows], color=cols, width=0.6)
for k, r in enumerate(rows):
    ax.text(k, 100 * r["val_acc"] + 0.05, f"{100 * r['val_acc']:.2f}", ha="center", fontsize=8, color=C.INK2)
lo = min(100 * r["val_acc"] for r in rows)
ax.set_ylim(lo - 1, 100.2)
ax.set_xticks(range(len(rows)), labels, fontsize=8)
ax.set_ylabel("Validation accuracy (%)")
ax.set_xlabel("Filters / dropout / augmentation (selected model in green)")
fig.tight_layout()
C.save(fig, "m01_tuning")
```

B.5 Occlusion sensitivity (step3_train_cnn.py)

```
p_va = model.predict(Xva, verbose=0)
base = p_va[np.arange(len(yva)), yva]
occ = np.zeros((8, 8))
occ_cls = np.zeros((10, 8, 8))
for r in range(8):
    for c in range(8):
        Xo = Xva.copy()
        Xo[:, r, c, 0] = 0.0                        # erase one pixel (one 4x4 block of the original page)
        drop = base - model.predict(Xo, verbose=0)[np.arange(len(yva)), yva]
        occ[r, c] = drop.mean()
        for d in range(10):
            occ_cls[d, r, c] = drop[yva == d].mean()
```

B.6 Test metrics and McNemar test (step4_evaluate.py)

```
def summary(prob):
    pr = prob.argmax(1)
    return {"accuracy": round(accuracy_score(y, pr), 4), "macro_f1": round(f1_score(y, pr, average="macro"), 4),
            "kappa": round(cohen_kappa_score(y, pr), 4), "log_loss": round(log_loss(y, np.clip(prob, 1e-7, 1)),
4),
            "macro_auc": round(roc_auc_score(y, prob, multi_class="ovr", average="macro"), 5),
```

```
            "errors": int((pr != y).sum())}


out["cnn"], out["knn"], out["logreg"] = summary(p), summary(P["p_knn"]), summary(P["p_lr"])
# McNemar test: CNN versus k-NN on the same test images
a_ok, b_ok = pred == y, P["p_knn"].argmax(1) == y
n01, n10 = int((a_ok & ~b_ok).sum()), int((~a_ok & b_ok).sum())
mc = stats.binomtest(min(n01, n10), n01 + n10, 0.5) if n01 + n10 else None
out["mcnemar_cnn_vs_knn"] = {"cnn_right_knn_wrong": n01, "cnn_wrong_knn_right": n10,
                             "p_value": float(mc.pvalue) if mc else 1.0}
acc_ci = stats.binomtest(int((pred == y).sum()), len(y)).proportion_ci(method="wilson")
out["cnn"]["acc_ci95"] = [round(acc_ci.low, 4), round(acc_ci.high, 4)]
```